

PYTHON – FOLHA DE CONSULTA

Valdemar W. Setzer

www.ime.usp.br/~vwsetzer (na seção [Recursos Educacionais](#)); ver a [versão em inglês](#), tradução desta
Versão 20.2 de 31/12/24

ÍNDICE

1. [Observações](#)
2. [Tipos, constantes, variáveis e matrizes](#)
3. [Operadores](#)
4. [Operadores lógicos \(booleanos\)](#)
5. [Funções nativas](#)
6. [Algumas funções e constantes matemáticas](#)
7. [Funções/métodos sobre listas](#)
8. [Precedência \(ordem de execução\)](#)
9. [Declaração e uso de uma função](#)
10. [Notação lambda](#)
11. [Identificadores globais e locais](#)
12. [Classes](#)
 - 12.1 [Introdução](#)
 - 12.2 [Polimorfismo](#)
 - 12.3 [Herança](#)
13. [Comandos compostos](#)
14. [Módulos](#)
 - 14.1 [Introdução](#)
 - 14.2 [Alguns módulos](#)
15. [Palavras reservadas](#)
16. [Referências](#)
 - 15.1 [Tutoriais](#)
 - 15.2 [Outras referências](#)
17. [Instalação do Python e uso do interpretador IDLE e seu editor](#)
18. [Curso](#)
19. [Textos, ambientes de programação, documentação e fóruns de programação](#)
20. [Agradecimentos](#)

1. OBSERVAÇÕES

1. Atenção: várias células das tabelas abaixo contêm uma única linha; se houver um célula com mais do que uma linha, aumente o tamanho da janela do navegador ou diminua o tamanho das letras (em geral, no PC com Ctrl –; para aumentar de volta, Ctrl +), para obter o alinhamento vertical adequado.
2. Todos os operadores, funções e comandos dos exemplos foram testados com o interpretador IDLE ("Integrated Development and Learning Environment"), as últimas com o Python 3.13.1, usando o Editor do IDLE (V. item 16.3 abaixo) a menos de alguns itens onde foi usado o Jupyter do Azure (ver o item [Ambientes](#)). Atenção: no IDLE, digitando uma linha e dando Enter o comando da linha é executado. Para dar vários comandos numa linha, eles devem ser separados por ;. Para inserir várias linhas antes de dar Enter, é necessário usar um editor de textos, copiar e colar no IDLE. Ou executar comandos que estão em um arquivo, ou editados com o Editor do IDLE.
3. Ver [página com informações sobre o IDLE](#). Em lugar do IDLE, pode-se ativar um compilador remoto da Python, fornecido pela W3 Schools. Basta ativar www.w3schools.com/python/trypython.asp?filename=demo_compiler, mas parece que ele não usa a última versão da Python. Ou, então, usar o Editor do IDLE, que é muito mais prático do que usar o IDLE (ver o item 16.3). No IDLE, ao dar Enter no fim de uma linha, ela é imediatamente executada. Os comandos compostos abrem linhas adicionais precedidas por Para inserir de uma vez múltiplas linhas antes de dar um Enter, é necessário usar um editor a copiar para o IDLE. Atenção: no Editor do IDLE, ou no compilador do W3, para exibir o resultado de alguma operação ou valor de variável, é necessário usar a função print(). Nos exemplos abaixo, o resultado da execução do uso direto do IDLE é indicado com a seta → sem que haja uma função print(). Se há um print() o teste foi feito em geral com o Editor do IDLE (V. 16.3).
4. Em todas as tabelas abaixo, supõe-se a execução prévia dos comandos de atribuição A=1; B=2; C=3; D=1.2; E=2.3; F=3.4; G='abc'; H='def'
5. Nas expressões usando operadores e funções, espaços em branco são ignorados. Assim, pode-se escrever A = 1; X= 2+ B; etc. Símbolos de operadores, nomes de funções, de variáveis e de comandos não podem conter espaços em branco. Assim, A b e + = não são considerados como a variável Ab e o operador +=, e sim como A, b, + e = separados (resultam em erros de sintaxe).
6. Comentários: # define o resto da linha como comentário (isto é, ignorado pelo interpretador ou compilador; "" ... "" define tudo entre os "" como comentário, inclusive várias linhas de código).
7. Nos exemplos, as palavras reservadas são grafadas em **negrito**.
8. Nos exemplos, após um comando para o IDLE em uma só linha, deve-se executá-lo digitando um Enter; o eventual resultado aparece neste texto depois de "→", p.ex. A+B → 3 (após dar-se o comando >>>A+B e Enter, aparece 3 na próxima linha). Atenção: ao usar copiar/colar podem ser coladas várias linhas, mas todas devem formar apenas um único comando, por exemplo um **if** ou **while** com várias linhas. No entanto, podem-se dar 2 ou mais comandos em uma só linha, separados por ";": X=1; Y=2; X,Y → (1,2)
9. Algumas funções exigem a adição de um módulo especial ao programa, o que é feito com o comando **import**. Para detalhes sobre ele, ver <https://docs.python.org/3/library/importlib.html>
10. Atenção: os exemplos desta página podem ser copiados e colados no Editor do IDLE (V. item 16 abaixo) e executados, cuidando-se para eliminar os →.
11. Ver uma página com [exemplos de programas completos](#) testados no ambiente Azure da Microsoft. Ver também [um site com muitos exemplos](#); os códigos estão nas abas "Test Suite".
12. Ver um excelente artigo sobre [estilo de programação em Python](#). {EF}
13. Esta folha de consulta usa Python básico. Não trata, por exemplo, de programação assíncrona.
14. Comentários, sugestões e críticas são muito bem vindos!!!
15. A seção 12 contém agradecimentos com o código dos colaboradores entre chaves {...}, mencionados nos trechos em que colaboraram.

2. TIPOS, CONSTANTES, VARIÁVEIS E MATRIZES

2.1 Tipos de variáveis

Em termos de tipos de variáveis, Python é considerada uma linguagem dinâmica e fortemente tipada. *Fortemente tipada*: Python não permite operações implícitas entre diferentes tipos sem conversão explícita. Por exemplo, adicionar uma cadeia de caracteres (*string*, ver 2.3 abaixo) a um número gerará um erro, a menos que a cadeia seja explicitamente convertida em um número por meio de uma função de conversão de tipos. *Dinamicamente tipada*: O tipo de uma variável é determinado em tempo de execução e pode mudar conforme novos valores são atribuídos a ela, o que significa que uma variável x adota o tipo do valor atribuído a ela. Por exemplo, ao executar a atribuição x = 1, x será do tipo inteiro. Se a atribuição x = 1.5 for executada, daí para diante x se tornará do tipo float, até mudar de tipo.

Variáveis funcionam como referências (ou ponteiros) para objetos. Quando um valor é atribuído a uma variável, ela não armazena diretamente o valor, mas sim uma referência ao objeto que contém o valor.

2.2 Tipos numéricos

Inteiro (int): precisão ilimitada. Ex.: 1234567890123456789012345

Constantes. Binária. Ex.: 0b101 ou 0B101(valor decimal 5); **octal**: 0o127 ou 0O127 (87); **hexadecimal**: 0xA5B ou 0XA5B (2651). Constantes hexadecimais são comumente usadas para representar números binários em uma notação mais fácil de ser lida.

Ponto flutuante (float): usualmente implementado usando o tipo double da linguagem C. Ex.: .12345678901234567890 → 0.12345678901234568 (note-se o

arredondamento).

Complexo: contém uma parte real, e uma imaginária indicada por um j . Exs.: constante $(1+2j)$ ou $(1+2J)$; variável $(A+D*1j)$

Para converter um tipo para inteiro usar as funções `int()`, para ponto flutuante `float()`, para complexo `complex()`. Essas funções devem ser usadas especialmente na entrada de dados, `input()` – que dá sempre um tipo `str` de cadeia de caracteres.

O módulo `NumPy` (ver as referências), que deve ser instalado, permite o uso de uma grande variedade de tipos numéricos. Para definir uma variável tipo ponto flutuante de 32 bits, basta dar, p. ex.

```
x = numpy.float32(1.0)
```

2.3 Tipo cadeia de caracteres (*string*)

Exemplo: `'tuv5xyz: ' → 'tuv5xyz: '; "tuv5xyz: " → 'tuv5xyz: '; 'tuv5x"yz: ' → 'tuv5x"yz: ';` Se é executado o comando `Cad='Esta é uma cadeia'` então `Cad → 'Esta é uma cadeia'`

Cadeia vazia: `"` (dois apóstrofes) `→ ""` ou `"" → ""`. Atenção: `" " → ' '` (um espaço em branco)

Indexação: o 1º índice indica o elemento inicial, começando em 0; o 2º o final, começando em 1: `CAD[0] → 'E'; Cad[5:10] → 'é uma'; Cad[10:] → 'é uma cadeia'`

Concatenação de cadeias: `'tuv5xyz: ' + Cad → 'tuv5xyz: Esta é uma cadeia'; 2*Cad → 'Esta é uma cadeiaEsta é uma cadeia'.`

2.4 Tipo *n*-pla ordenada (*tuple*)

Exemplo: `Tup = (A, 5, 3.14, 'bla');` `Tup → (1, 5, 3.14, 'bla')`. (Para o valor de A ver o item 4 de 1. Observações.)

n-pla vazia: `()`

Indexação: `Tup[0] → 1; Tup[1:3] → (5, 3.14); Tup[2:] → ((3.14, 'bla');` `2*Tup → (1, 5, 3.14, 'bla', 1, 5, 3.14, 'bla')`.

É *inválido* atribuir um valor a um elemento de uma tupla: `Tup[1] = 10`

2.5 Tipo lista (*list*)

Exemplo: `L = [A, 5, 1.2, 'bla']`

Lista vazia: `[]`

Indexação: `L[2] → 1.2; L[1:4] → [5, 1.2, 'bla']`. Note-se que os índices dos elementos vão de 0 a, digamos, n . `L[i:j]` indica os elementos começando no elemento de índice i e terminando no de índice $j-1$. `L[5] →` dá um erro pois não há elemento de índice 5. `L[4] = 9 →` dá o mesmo erro. Para adicionar mais um elemento (sempre no fim da lista): `L = L + [9]; L → [1, 5, 1.2, 'bla', 9]; L = L + []` não altera L.

Atribuição a um elemento: é *válido* fazer `L[3]=10; L → [1, 5, 1.2, 10]`.

Eliminação de um elemento da lista: usa-se o comando **del**. Ex. **del** `L[2]; L → [1, 5, 10];`

2.6 Tipo dicionário (*dictionary*)

Exemplo: `Dic = {5:10, 3:'bla', 'ble':'A', 'A':'8'}`; `Dic → {5: 10, 3: 'bla', 'ble': 'A', 'A': 8}`

Indexação: `Dic[5] → 10; Dic[3] → 'bla'; Dic['ble'] → 1; Dic[3] → 'bla'; DIC['A'] → '8'`

Note-se que cada elemento de um dicionário é da forma $x:y$, onde x é o índice e y é o valor associado a esse índice. Índices ou valores que são alfabéticos devem estar entre apóstrofos `'...'` ou aspas `"..."`.

Obtenção de **todos os índices** (*keys*): `Dic.keys() → dict_keys([1, 'ble', 3, 5])`

Obtenção de **todos os valores**: `Dic.values() → dict_values([8, 1, 'bla', 10])`

`dict_keys` e `dict_values` não podem ser indexadas. Para se trabalhar com todos os índices ou os valores, pode-se transformá-los em listas e depois trabalhar com elas {MDG}:
`L = [x for x in Dic.keys()]; L → [5, 3, 'ble', 1]`
`L = [x for x in Dic.values()]; L → [10, 'bla', 1, 8]`

2.7 Tipo conjunto (*set*)

Exemplo: `S = {1, 'dois', 3, 'quatro'}`; `S → {3, 1, 'dois', 'quatro'}` (Conjuntos não são ordenados e não podem ser indexados.)

Conjunto vazio: `{}`

Usos. *Número de elementos* (*cardinalidade*): `len(S) → 4`. *Pertinência*: `'dois' in S → True`. *União*: `S | {5} → {1, 3, 'dois', 5, 'quatro'}` (S não mudou!).

Interseção: `S & {1, 'dois'} → {1, 'dois'}`. *Complementação*: `S - {1, 'dois'} → {3, 'quatro'}`. Testa *superconjunto próprio*: `S > {1, 'quatro'} → True`.

Superconjunto: `>=`. *Subconjunto próprio*: `<`. *Subconjunto*: `<=`. *União exclusiva* (elimina os elementos comuns): `S ^ {'dois', 5} → {1, 3, 5, 'quatro'}`

Há dois tipos de conjuntos: *set*, em que seus elementos podem ser mudados, e *frozenset* que não pode ter seus elementos mudado. A construção de um *set* é automática. Ex.: `x = frozenset({1, 2}); x.pop() → erro`. Conjuntos internos a conjuntos têm que ser do tipo *frozenset*.

2.8 Tipo lista de intervalo de inteiros (*range*)

Padrão: `range(m, n, k)`, com n e k opcionais ou só k opcional, gerando os inteiros de uma lista r tal que $r[i] = m + k*i$, onde $i \geq 0$ e $i < \text{abs}(n)$

Exemplos: `list(range(5)) → [0, 1, 2, 3, 4]; list(range(1, 5)) → [1, 2, 3, 4]; list(range(0, 8, 2)) → [0, 2, 4, 6]; list(range(-1, -10, -2)) → [-1, -3, -5, -7, -9]`

Uso em comandos **for**: ver o comando no item 11 abaixo e no 2.5 acima.

2.9. Vetores e matrizes (*arrays*)

Ao contrário de quase todas as outras linguagens de programação, Python não tem o tipo *array*, pois os valores de variáveis podem ocupar tamanhos diversos. Para representar matrizes, usam-se listas, já que as mesmas podem ser indexadas como se fossem matrizes. Em outras linguagens, a declaração de um array produz a inicialização de seus valores, isto é, a declaração do array faz com que todos seus elementos passem a existir. Mas o mesmo não ocorre com as listas de Python; assim, é preciso adicionar cada elemento novo, pela ordem. Para usar matrizes, é necessário carregar o módulo `NumPy`, que permite o uso de vetores e matrizes como se fosse nas linguagens tradicionais. Supondo que ele está instalado, para ativá-lo e dar-lhe o nome interno de np dá-se **import** `NumPy as np`. Ver as referências para detalhes como usá-lo.

Exemplos

1. Para definir um vetor de 5 elementos, inicializando com valores 0 (poderia ser outro qualquer, como I ou uma expressão): `V=[0 for I in range(4)]; V → [0, 0, 0, 0];` note-se que `V[4]` não existe e não pode receber algum valor, como `V[4]=0` ou `V[4]=[0]`. Para defini-lo: `V=V+[0]`. Para varrer todo um vetor de tamanho variável, use `len(V) → 4`

2. Para definir uma matriz bidimensional, constrói-se uma lista de listas: `M=[[1, 2], [3, 4]];` `M → [[1, 2], [3, 4]];` `M[1][1] → 4;` `M[0][1] → 2;` `M[1][0] → 3`. Inicialização, com vários valores, de uma matriz com 3 linhas e 4 colunas:
`M = [[I+J+1 for I in range(2)] for J in range(3)]; M → [[1, 2], [2, 3], [3, 4]];` `M[2][1] → 4`

Para dimensões maiores, basta estender as receitas.

3. Para varrer todos elementos e exibir os seus valores: M=[[1, 2], [3, 4]] for I in M: for E in I: print (E) → 1 2 3 4	4. Para gerar uma matriz com valores crescentes: M=[] for I in range(2): M[I]=M[I]+[-1] M → [-1, -1] J=1 for I in range(2): M[I]=[J,J+1]
--	---

5. Para obter linhas: for I in M: I → [1, 2] [3, 4]	J=J+2 M → [[1, 2], [3, 4]] {PC} Chamou a atenção para a inicialização do J
6. Para obter colunas: J=0 J =[I[J] for I in [M] J → [1, 3]	7. Para aplicar uma função, p.ex. sqrt a todos os elementos de M: [[math.sqrt(x) for x in I] for I in M] → [[1.0, 1.4142135623730951], [1.7320508075688772, 2.0]]

2.10 Iteráveis

Em Python, um *iterável* (*iterable*) é um objeto capaz ao qual se pode fazer acesso a seus membros um de cada vez, permitindo que ele seja varrido em um comando de malha de repetição **for**. Exemplos comuns de iteráveis incluem listas, tuplas, dicionários e conjuntos.

3. OPERADORES

Op	Significado e tipos dos operandos	Exemplos
+	Operador binário (com dois argumentos): soma de int, float, ou complex; concatenação de cadeias de caracteres (<i>strings</i>), de listas e de <i>n</i> -plas	A+B → 3, D+E → 3.5; A+D → 2.2; (A+D*1j)+(E+C*1j) → (3.3+4.2j); G+H → 'abcdef'; (123, 'xyz') + ('L', '3.14') → (123, 'xyz', 'L', 3.14) [1, 2] + [3] → [1, 2, 3]
+	Operador unário (com um argumento): sem efeito	+A → 1
-	Operador binário de subtração de int, float ou complex; complementação de conjuntos	C-B → 1; F-D → 2.2; F-A → 2.4
-	Operador unário de troca de sinal	-A → -1
*	Multiplicação int, float ou complex	B*C → 6; D*E → 2.76; B*D → 2.4; (A+D*1j)+(E+C*1j) → (3.3+4.2j)
/	Divisão int, float, resulta float ou complex	C/B → 1.5; F/E → 1.4782608695652175; C/D → 2.5; (A+D*1j)/(E+C*1j) → (0.41287613715885235-0.016794961511546552j)
//	Divisão de int, resulta int, ou de float por int ou float só parte inteira	C//B → 1; F//D → 2.0; F//B → 1.0;
%	Resto int da divisão de ints, parte inteira se divisão de floats	C%B → 1; F//D → 2.0; F//B → 1.0
**	Potenciação de int, float ou complex	B**C → 8; B**D → 2.2973967099940698; D**B → 1.44; D**E → 1.5209567545525315; (1+1j)**2 → 2j; 27**(1/3) → 3 (raiz cúbica)
==	Testa igualdade de int, float complex ou string, resulta True (verdadeiro) ou False (ver tabela de operadores lógicos); com vários == em um só comando dá True somente se cada um dos pares consecutivos for igual	B==A*2 → True; A==B → False; A==D → False; A==int(D) → True; (A+B*1j)==(1+2j) → True; G=='abc' → True; G==H → False; 1==1==1 → True; 1==1==2 → False;
!=	Diferente, idem	A!=B → True; A!=D → True; A!=int(D) → False; (1+2j)!= (2+2j) → True; G!=H → True; 1!=2!=3 → True; 1!=1!= 2 → False;
>	Maior do que, idem sem complex; testa superconjunto próprio	B>A → True; A>B → False; D>A → True; E>D → True; H>G → True; 3>2>1 → True; 3>2>3 → False;
<	Menor do que, idem; subconjunto próprio	A<B → True; B<A → False; etc.
>=	Maior ou igual, idem; superconjunto	B>=A → True; B>=D → True; H>=G True; etc.
<=	Menor ou igual, idem; subconjunto	B<=A → False; etc.
&	"e" bit a bit de valores binários, resultado decimal no IDLE; intersecção de conjuntos	0b0101 & 0b0001 → 1; bin(0b1100 & 0b1010) → '0b1010' (na exibição corta os 0s à esquerda)
	"ou inclusivo" bit a bit, resultado decimal no IDLE; união de conjuntos	bin(0b1100 0b1010) → '0b1110'
^	"ou exclusivo" bit a bit. resultado decimal no IDLE; união exclusiva de conjuntos	bin(0b0110 ^ 0b1010) → '0b1100'
>>	deslocamento bit a bit para a direita inserindo 0s à esquerda (equivale a divisão por pow(2,n)	J = 10; bin(J) → '0b1010'; bin(J>>1) → '0b101' (zeros à esquerda não são exibidos)
<<	deslocamento bit a bit para a esquerda inserindo 0s à direita (equivale a divisão por pow(2,n)	J = 10; bin(J) → '0b1010'; bin(J<<2) → '0b101000'
is	Teste de identidade de objetos	
is not	Teste de não identidade de objetos	
=	Atribuição simples ou múltipla, lado direito int, float, complex ou string; ambos os lados podem ser uma <i>n</i> -pla sem "(" e ")"	A=1; A → 1; A=D; A → 1.2 (A mudou de int para float); J=(A+D*1j); J → (1+1.2j); J, I, K = 10, 20, B*30; I → 20; J, K → (10, 60) Troca de valores de variáveis: X, Y= 3, 4; X, Y= Y, X; X, Y → (4, 3)
+=	x += y equivale a x = x + y	J=1; J+=2; J → 3; J=(1+2j); J+=(2+3j); J → (3+5j)
-=	x -= y equivale a x = x - y, inclusive de conjuntos	J=3; J-=2; J → 1
*=	x *= y equivale a x = x * y	J=2; J*=3; J → 6
/=	x /= y equivale a x = x /y	J=6; J/=3; J → 2.0
//=	x //= y equivale a x = x // y	J=15; J//=4; J → 3; J=15.5; J//=3.7; J → 4.0
%=	x %= y equivale a x = x%y	J=6; J%=4; J → 2.0; J=6; J%=2; J → 0
=	x **= y equivale a x = xy	J=2; J**=3; J → 8 (Não disponível na versão 2 do Python)
>>=	x >>= y equivale a x = x>>y	J=0b1010; J>>=2; bin(J) → '0b10' (zeros à esquerda não são exibidos)
<<=	x <<= y equivale a x = x<<y	J=0b1010; J<<=2; bin(J) → '0b101000'
&=	x &= y equivale a x = x&y, inclusive de conjuntos	J=0b1100; K=0b1010; J&=K; bin(J) → '0b1000'
^=	x ^= y equivale a x = x^y, inclusive de conjuntos	J=0b1100; K=0b0110; J^=K; bin(J) → '0b1010'
=	x = y equivale a x = x y, inclusive de conjuntos	J=0b1100; K=0b1010; J =K; bin(J) → '0b1110'
if	Expressão condicional: x = Valor-se-True if Expressão-Lógica else Valor-se-	J = 1 if 4>3 else 2 → 1 J = 5 * (1 if B<A else 3*A); → 15

4. OPERADORES LÓGICOS (BOOLEANOS)

Op	Significado	Exemplos
	No que segue, supõe-se a execução prévia de L1T = True ; L2T = True ; L1F = False ; L2F = False	
True	Constante indicando "verdadeiro"	Atenção: true não é aceito, dá erro de variável não definida
False	Constante indicando "falso"	Idem para false; são considerados como False : None , 0, 0.0, 0j, ' ', (), [], {};
not	Negação: muda True para False e vice-versa	not L1T → False; not L1F → True
x or y	"ou" inclusivo; dá falso somente se x e y forem falsos	L1T or L2T → True; L1T or L1F → True; L1F or L1T → True; L1F or L2F → False
x and y	"e"; dá verdadeiro somente se x e y forem verdadeiros	L1T and L2T → True; L1T and L1F → False; L1F and L1T → False; L1F and L2F → False
x in y	se x está na <i>string</i> , <i>n</i> -pla, lista ou conjunto y dá True, senão False	'a' in 'false' → True; 5 in (2, 5, 3) → True; 3 in [2, 5, 3] → True; 4 in [2, 5, 3] → False
x not in y	Contrário de in	'a' not in 'false' → False; 5 not in (2, 5, 3) → False; 3 not in [2, 5, 3] → False; 4 not in [2, 5, 3] → True
is	Teste de identidade de dois objetos. Determina se duas variáveis apontam para o mesmo objeto.	x = y = 0; x is y → True; ; x = 0; y = 1; print(x is y) → False
is not	Teste de identidade de dois objetos. Determina se duas variáveis não apontam para o mesmo objeto.	x = y = 0; x is not y → False; x = 0; y = 1; print(x is not 1) → True

5. FUNÇÕES NATIVAS (Ver a lista oficial em <https://docs.python.org/3/library/functions.html> ; os exemplos foram testados com o IDLE ou com o Editor do IDLE; nesse último caso a exibição de resultados é feita com Print()).

Função	Significado	Exemplos
abs()	Valor absoluto; módulo no caso de complexo	abs(-1) → 1; abs(2) → 2; abs((1+2j)) → 2.23606797749979
S.add(x)	Insere x no iterável (V. item 2.10) S	S = {1, 2, 'três'}; S.add('quatro'); S → {1, 2, 'três', 'quatro'} Aparentemente, conjuntos são exibidos em ordem alfabética
aiter()	Cria um iterável assíncrono. Usada em programação assíncrona (permite execução concorrente, sem esperar que operações sejam completadas). A programação normal é síncrona, uma operação de cada vez, sequencialmente	Exige a importação de módulos específicos, como o asyncio para entrada/saída. Ver exemplos.
all(x)	Retorna True se todos elementos do iterável x tiverem valor True ou se x for vazio, e False se algum elemento de x for False	a = [True, True, True]; b = [True, True, False]; c = []; all(a), all(b), all(c) → (True, False, True)
anext()	Usada em programação assíncrona (v. aiter). Permite obter o próximo item de um iterável assíncrono	Parte do módulo asyncio. Ver exemplo.
any()	Retorna True se qualquer elemento for True e False se estiver vazio.	a = [True, True, True]; b = [True, True, False]; c = []; any(a), any(b), any(c) ? (True, True, False)
ascii(x)	Retorna x como cadeia, pulando caracteres que não são do tipo ASCII {EF}	numero = 123; list = [1, 2, "Olá"]; tuple = ("a", "b", "c") print(ascii(numero), ascii(list), ascii(tuple)) → 123 [1, 2, 'Ol\xe1'] ('a', 'b', 'c')
bin()	Converte um int em binário	bin(B) → '0b10'; bin(20) → '0b10100'
x.bit_length()	Comprimento de bits significativos do binário x	0b101010.bit_length() → 6; 0b001010.bit_length() → 4
bool()	Retorna True se o argumento é verdadeiro, False se não há argumento ou ele é falso	bool(B>A) → True; bool(C>D) → False; bool() → False
breakpoint() #msg	Interrompe a execução, emitindo a mensagem msg, e ativa o depurador (<i>debugger</i>) pdb.	for i in range(5): if i == 2: breakpoint() # Interrupção aqui print(i) → 0 1 -> breakpoint() # Interrupção aqui (Pdb)
bytearray(x,c,er)	Retorna um objeto bytearray que é uma matriz dos bytes do iterável x usando o código de tipo c, utf-8 em hexadecimal se c é ausente; er é uma cadeia com uma mensagem de erro se algum caractere não pode ser convertido	prime = [2, 3, 5, 7]; byte = bytearray(prime); print(byte) → bytearray(b'\x02\x03\x05\x07')
bytes(x,c,er)	O mesmo que bytearray mas o resultado é um objeto que não pode ser modificado	for i in range(10): if i == 6: breakpoint() # Pause execution here print(i)
callable()	Retorna True se o argumento dessa função for uma função já definida, False caso contrário.	x = 10; print(callable(x)) → False def func(x): return (x); y = func; print(callable(y)) → True
chr()	Caractere correspondente ao código ASCII do argumento (entre 0 e 255)	chr(97) → 'a'; chr(150) → '\x96' (converteu para hexadecimal, pois não achou o caractere)
classmethod()	Permite ativar diretamente o método de uma classe, sem precisar instanciar a classe {EF}	class CM: a = 0 def mostra_a(cls, x): cls.a = x

		<pre>print("a" obtido:', cls.a) CM.mostra_a = classmethod(CM.mostra_a); CM.mostra_a(123) → "a" obtido: 123</pre>
x.clear()	Remove todos os elementos do iterável x (V. 2.10)	<pre>x = {1, 2, 'três'}; x.clear(); x → set() [indica conjunto vazio]; x != {9}; x → {9}</pre>
compile(s,file,mo)	Permite compilar e executar um objeto s (p.ex. uma <i>string</i>) retornando o resultado se o modo for 'exec', podendo ser 'eval' se s for uma expressão, 'single' se contém um comando iterativo; file se s está em um arquivo {EF}	<pre>string = 'a=8;b=7;soma=a+b\nprint(soma)' # a=8;b=7;print(a+b) ref = compile(string, ' ', 'exec'); exec(ref) → 15</pre>
complex(re,im)	Converte em complexo com parte real re e imag. im	<pre>complex(1) → (1+0j); complex(2.5) → (2.5+0j)</pre>
x.conjugate()	Dá o conjugado do complexo x	<pre>x = (1+2j); x.conjugate() → (1-2j) [não está funcionando na versão 3.13.1]</pre>
delattr(cl,atr)	Elimina o atributo atr da classe cl	<pre>class A: x = 10; y = 20 inst = A(); print(inst.x, inst.y); delattr(A, 'x'); print (inst.y); print(inst.x) → 10 20 20 AttributeError: 'A' object has no attribute 'x'</pre>
dict()	Cria um dicionário {EF}	<pre>n = dict(x=5, y=0); print(n, type(n)) → {'x': 5, 'y': 0} <class 'dict'></pre>
dir()	No IDLE lista o nome das variáveis da sessão	<pre>n = [10]; print(dir()) → [..., n]</pre>
x.discard(y)	Remove o elemento y do conjunto x, se y está em x	<pre>x = {1, 2, 'três'}; x.discard('três'); x → {1, 2}</pre>
divmod(x,y)	Dá a dupla ordenada (x // y, x % y)	<pre>divmod (C,B) → (1,1), divmod (C,D) → (2.0, 0.6000000000000001)</pre>
x.encode(c)	Codifica a cadeia x usando o tipo de código c	
enumerate(x)	Retorna o iterável (V. 2.10) x enumerando cada um de seus elementos {EF}	<pre>lis = ['a', 'b', 'c']; print(list(enumerate(lis))) → [(0, 'a'), (1, 'b'), (2, 'c')]</pre>
eval(x)	Executa a cadeia (<i>string</i>) x e retorna seu valor. x pode ser o resultado de uma input(). x não pode ser um comando composto.	<pre>eval('2*3') → 6 eval("abc'[1]") → 'b'</pre>
exec('c',gl,loc)	Executa imediatamente o código c; gl e loc são dicionários permitindo especificar variáveis e métodos globais e locais a serem usados, garantindo segurança	<pre>exec('print("A soma de 5 e 10 é", (5+10))') &rrar A soma de 5 e 10 é 15 exec('x = 10\ny = 20\nprint("Sum:", x + y)') → Sum: 30 Em lugar de \n (indica nova linha) pode-se usar ;</pre>
exit()	Termina a execução do programa	Requer ométodo sys, incorporado com import sys. Uso: sys.exit
filter(f, L)	Aplica a função f a cada elemento da lista L, e resulta nos elementos de L para os quais f for True	Ver exemplo no item "notação lambda"
float()	Converte para float	<pre>float(B) → 2.0;</pre>
float.as_integer_ratio()	Dá um par de inteiros cuja razão é o argumento	<pre>float.as_integer_ratio(1.5) → (3,2)</pre>
float.is_integer()	Dá True se o argumento for inteiro, False em caso contrário	<pre>float.is_integer(1.5) → False; float.is_integer(3.0) → True</pre>
format(x,f)	Retorna valor de x no formato f. Ver a grande lista de formatos {EF}	<pre>a = 123; print(format(a, 'x')); print(format(a, 'b')) → 7b 1111011 Obs.: 'x' dá hexa</pre>
frozenset()	Constrói um conjunto que não pode ser mudado	<pre>x = frozenset({1, 2}); x → {1, 2}; x.pop() dá erro</pre>
getattr(obj,atr,err)	Retorna o valor de um atributo atr do objeto obj. Se o atributo não existe em obj, retorna err.	<pre>class Cla: def __init__(self, X, Y): self.X = X self.Y = Y cla1 = Cla(13, 'abc') #Creates an instantiation cla2 = Cla(15, 'def'); print(getattr(cla1, 'X', getattr(cla2, 'Y'))) → 13 def</pre>
globals()	Retorna um dicionário com os nomes e valores de todas as variáveis globais {EF}	<pre>a = 1; b = 2; print(globals()) → {..., 'a': 1, 'b': 2}</pre>
hasattr(clobj,atr)	Retorna True se a classe ou clobj tem o atributo atr {EF}	<pre>class A: a = 123 obj = A() #cria o objeto obj print(hasattr(A, "a")); print(hasattr(obj, "b")) → True False</pre>
hash(x)	Retorna um inteiro único que representa x, que pode ser apenas uma <i>string</i> , um número ou uma <i>n-pla</i> . O valor pode mudar de uma sessão para outra	<pre>hash('Bom dia!') → -2257218869280192840 hash((1,2,3)) → 529344067295497451</pre>
help()	No modo interativo (IDLE), dá a documentação do argumento	<pre>help(int)</pre>
hex()	Converte um int para um hexadecimal	<pre>hex (8) → '0x8'; hex (50) → '0x32'; hex(C) → '0x3'</pre>
id(x)	Quando o objeto x é criado, é associado um número (endereço) único a x, que é retornado pela função {EF}	<pre>a = 5; b = 6; print('a', id(a), 'b' id(b), '123' id(123) → a 140720283321384 b 140720283321416 c 140720283321576 123 140720283325160</pre>
__import__()	Importação de módulos. É o mesmo que o comando import .	Ver https://docs.python.org/3/library/importlib.html
input("mensagem")	Espera o usuário dar uma entrada, seguida de Enter . Retorna o valor da entrada de um dado no tipo <i>string</i> ; pode exibir mensagem ao ser executado. Não funcionou no compilador <i>on-line</i> W3 (item 1-3)	<pre>dia=input('Entre com o valor do dia:'); → "Entre com o valor do dia:" 20 Enter dia → '20' (como <i>string</i>); para calcular é preciso converter, p.ex. diaint=int(input('Entre...')) ou diaint=int(dia); diaint → 20</pre>
int()	Converte para int	<pre>int(D) → 1; int('123') → 123</pre>
is not	Teste de não identidade objetos. Determina se duas variáveis apontam para objetos diferentes.	<pre>x = y = 0; x is not y → False</pre>
x.isdisjoint(y)	True se o conjunto x é disjunto do conjunto y, False em caso contrário	<pre>{1, 2, 'três'}.isdisjoint({4, 'cinco'}) → True {1, 2, 'três'}.isdisjoint({2, 'cinco'}) → False</pre>

isinstance(clobj,t)	Retorna True se a classe ou objeto clobj é do mesmo tipo que t; se t é uma n-pla, retorna True se obj é um dos tipos da tupla {EF}	print (isinstance("Olá", (float, int, str, list, dict, tuple))) → True
issubclass(c1,c2)	Retorna True se a classe c1 for uma subclasse da classe c2, False em caso contrário{EF}	class A: a = 1 class B(A): b = 2; print(A.a, b) print(issubclass(B,A)) → 1 2 True
iter(x)	Retorna um índice para os elementos do iterável (V. 2.10) x {EF}	x = iter(["a", "b", "c"]) print(next(x),next(x),next(x)) → a b c
s.join(x)	Método da classe string. Concatena as partes da lista, tupla ou conjunto x, separando-as com a <i>string</i> s.	SeparaComVirgulas = ','.join(['a', 'b', 'c']); SeparaComVirgulas → 'a,b,c'
len(x)	Dá o número de elementos de um iterável (V. 2.10) – neste caso, a cardinalidade	len(G) → 3; len((1,2,3,4)) → 4; len((A,B,G)) → 3; len([1,B,5,7]) → 4 len({1, 2, 'três'}) → 3
list()	Converte os elementos de um iterável em lista; sem argumento dá a lista vazia	list(G) → ['a', 'b', 'c']; list((1, 2, 3)) → [1, 2, 3]; list({1,2,3}) → [1, 2, 3]
locals()	Retorna um dicionário com os nomes e valores de variáveis locais a um programa ou função	def f(x): x = 10 print(locals()) return (x+3) print(f(20)) → {'x': 10} 13
lower()	Converte as letras uma <i>string</i> para minúsculas	'BLA3#'.lower() → 'bla3#'; nome = input('Digite seu nome:').lower(); print(nome)
s.lstrip('c')	Método da classe string. Elimina todos os caracteres c do começo da <i>string</i> s, ignorando brancos até c; se c é omitido, elimina os brancos no começo de s	s='x y z'; s.lstrip('x') → s=(' x y z '); s.lstrip('x') → ' y z'; s.lstrip() → 'x y z'; s=(' x y z '); s.lstrip() → 'x y z '; s.lstrip(' x,') → 'y z '; s.lstrip('x y') → 'z '
map(f,L)	Aplica a função f a cada elemento de um ou mais iteráveis (V. 2.10) L	list (map (abs, [2,-3,4,-5])) → [2, 3, 4, 5]
max(x)	Dá o maior dos elementos do iterável x	max (1,2,3) → 3; max (['a', 'b', 'c']) → 'c'; max ('a', 'b', 'c') → 'c'; max (G) → 'c'
memoryview(x)	Retorna um objeto com com a representação interna de x {EF}	a = bytearray('ABC', 'utf-8'); m = memoryview(a); print(list(m[1:3])) → [66, 67]
min()	Como max, para o menor	min (1,2,3) → 1; min (['a', 'b', 'c']) → 'a'; min ('a', 'aa', 'aaa') → a; min (G) → 'a'
next(x)	Retorna o próximo valor do iterável x {EF}	a = [5, 6, 7]; b = iter(a); print(next(b), next(b)) → 5 6
object()	Retorna a classe que é a base de todas as classes; não se pode definir atributos a ela, mas ela tem os atributos comuns a todas as classes, como <code>__init__</code> e <code>__string__</code> (EF)	obj = object(); print(dir(obj)) → ['__class__', '__delattr__', '__dir__', '__doc__', ...]
oct()	Converte para octal	oct(15) → '0o17'
open(arq,m)	Abre o arquivo de caminho e nome arq tornando-o um objeto, no modo m, que pode ser r (somente leitura), w (gravação, cria arq se não existe), a (adiciona uma cadeia a arq, cria se não existe), x (cria arq, erro se já existe), b (arquivo binário, como p.ex. de imagem), t (arquivo de texto, padrão). arq torna-se uma classe, com atributos	f = open('nome', 'w'); f.write('Texto'); f = open('nome', 'r') print(f.read()); f.close() → Texto
ord()	Contrário de chr	ord ('a') → 97
x.pop()	Returns iterable x without its last element	x = {1, 2, 'three'}; x.pop(); x → {1,2}
pow(x,y)	Equivalent to x**y	pow(2,3) → 8; pow(4,0.5) → 2.0; pow (4,-2) → 0.625
print(x)	Saída de dados. x pode conter vários objetos e expressões separados por vírgulas, bem como cadeias a serem exibidas como constantes, e parâmetros de formato de linha (\...)	print(A,D) → 1 1.2; print ('A =',A) → A = 1; print ('A*3 =',A*3,'\nD =',D) → A*3 = 3 D = 1.2
property(cl)	Usada dentro de uma classe cl, permite alterar os valores dos atributos de cl {EF}	See examples here
random	Uma classe. Exige importar o módulo random. Algumas funções dessa classe: 1. random.randint(a,b) retorna inteiro pseudo-aleatório entre a e b inclusive. 2. random.random() dá o próximo aleatório em ponto flutuante entre 0.0 e 1.0 3. random.uniform(a, b) idem entre a e b inclusive 4. random.choice(x) dá aleatoriamente um elemento da lista x não vazia 5. random.shuffle(x) ordena a lista x aleatoriamente	Os resultados obtidos com o IDLE podem ser outros, dependendo da "semente": 1. import random; random.randint(10, 100) → 12; random.randint(10, 100) → 67 2. random.random() → 0.05462293624556047; random.random() → 0.36903357168070205 3. random.uniform(2,5) → 3.983840861586745 4. random.choice([1,2,3,4,5]) → 3; random.choice([1,2,3,4,5]) → 4 5. x=[1,2,3,4]; random.shuffle(x); x → [3, 1, 4, 2]; random.shuffle(x); x → [4, 1, 2, 3]
range() range(in,fim,passo)	Cria virtualmente uma lista virtual, para ser usada em um comando for . in indica a ordem do primeiro elemento, fim a ordem do último, e passo (se omitido, fica 1) os incrementos na ordem.	range(C): equivale a [0, 1, 2]; range(1, 5) a [1, 2, 3, 4, 5]; range(0, 10, 3) a [0, 3, 6, 9]; range(0, -4, -1) a [0, -1, -2, -3]
reduce(fu,li)	Essa função é parte do módulo operator. Ela aplica a função fu a todos elementos da lista li. Requer incorporar o módulo functools, permitindo que operadores (itens 3 e 4 acima) sejam especificados como funções. Uma outra possibilidade é usar a notação	import functools import operator #ver operadores em https://docs.python.org/3/library/operator.html lis = [1,3,5]; functools.reduce(operator.add, lis) → 9

	lambda (item 10 abaixo), definindo uma função com um operador.	functools.reduce(lambda x, y: x+y, lis) #estende o + para todos os members da lista 9
reload(x)	Reloads a function x defined in a present module	Look for examples
repr()	Retorna o valor de x em forma de <i>string</i> . x pode ser um objeto de uma classe, criando-o. Útil para depuração. (Procure por exemplos com classes.)	x = "abc"; y= [1, 2, 3]; print(repr(x, repr(z)) → ("abc", '[1, 2, 3]')
reversed(x)	Retorna o iterável (V. 2.10) x em sua ordem reversa. Não se aplica a sets {EF}	print(list((reversed('abc')))); print(list((reversed(('d','e','f')))) → ['c', 'b', 'a'] ['f', 'e', 'd'])
round(x,n)	x arredondado na n-ésima casa decimal; sem n arredonda para o inteiro	round(3.5566,3) → 3.557; round(4.5555,3) → 4.555; round(3.5555) → 4; round(3.4555) → 3
s.rstrip(c)	Idem a lstrip, elimina a string c à direita da string s	print("exemplo!!!".rstrip("!"), "exemplo!!! ".rstrip("!")) → exemplo exemplo!!! {EF}
set()	Constroi um conjunto que pode ser mudado	set([1, 2, 'three']) → {1, 2, 'three'}; set() → set()
setattr(cl, 'atr', val)	Define um valor val para o atributo atr da classe cl {EF}	class Person: name = 'Eva'; age = 36 setattr(Person, 'age', 25); print(Person.age)
slice(in,fim,passo)	Define índices de um iterável para ser usado posteriormente para extrair parte do iterável. in, fim e passo como na função range()	T = ("a", "b", "c", "d", "e", "f"); y = slice(2); T[y] → ('a', 'b') z = slice(3, 5); T[z] → ('d', 'f') cad = 'abcdefghi'; t = slice(5, 7); cad[t]; → 'fgh'
sorted()	Ordena uma lista	sorted ([1,4,2]) → [1, 2, 4]; sorted([B,A]) → [1,2]
staticmethod(cl, m)	Converte um método m de uma classe cl em um método estático, que pode ser usado sem se instanciar a classe em um objeto da mesma	class Cl: def ad(a,b): return a + b Cl.ad = staticmethod(Cl.ad) print(Cl.ad(5,7)) #Ativação da função ad sem criar um objeto de Cl → 12
str()	Converte int ou float para string	str(C) → '3'; str(D) → '1.2'
c.strip(s)	Método da classe string. Dá a <i>string</i> s sem a <i>string</i> c no começo e no fim. Sem o c, considera brancos. Ver também lstrip e rstrip	s = ' x y z '; s.strip() → 'x y z'; s = 'xyxzyzzxyxy'; s.strip('xy') → 'zzz'
sum()	Soma os elementos de uma lista, n-pla ou conjunto	sum([A,B,C, 4, D]) → 11.2; sum((1,2,3)) → 6; sum ({1, 2, 3}) → 6
super()	Permite que uma classe C2 que herda as propriedades de uma classe C1 utilize os atributos dessa última	class C1: #Exemplo baseado em um da W3 Schools def __init__(self, txt): self.mensagem = txt def printmensagem(self): print(self.mensagem) class C2(C1): def __init__(self, txt): super().__init__(txt) x = C2("Olá!"); x.printmesagem() → Olá!
time.sleep(n)	Interrompe a execução de um programa por n segundos. Exige a carga do módulo time: import time	import time; print(10); time.sleep(5); print(20) → 10 [pause] 20 Não funciona no compilador <i>on-line</i> da W3 School; funciona no IDLE
tuple()	Converte um iterável em uma lista ordenada	tuple('abc') → ('a', 'b', 'c'); tuple([1, 2, 3]) → (1, 2, 3); tuple({1,2,3}) → (1, 2, 3)
type()	Se o argumento for uma variável, dá seu tipo; se for um objeto, o tipo do mesmo. Pode ser usado para criar uma classe dinamicamente.	type (A) → <class 'int'>; type (D) → <class 'float'>; type (G) → <class 'str'>
upper()	Converte as letras de uma <i>string</i> para maiúsculas	'bla3#'.upper() → 'BLA3#'
vars(obj)	Dá o atributo __dict__ do objeto obj. Permite ver os atributos de obj em forma de dicionário. Outros atributos usados pelo sistema também aparecem	class Ex: X = "abc" Y = 12 atrs = vars(Ex) {'X':"abc", 'Y':12}
zip()	x deve ser um número de iteráveis em paralelo, em princípio com mesmo número de elementos em cada iterável. O resultado será de duplas, triplas etc., juntando elementos correspondentes dos iteráveis. É como se transformasse linhas em colunas.	for I in zip([1, 2, 3], ['a', 'b', 'c']): print(I) → (1, 'a') (2, 'b') (3, 'c')

6. ALGUMAS FUNÇÕES E CONSTANTES MATEMÁTICAS

Para usar essas funções, é necessário executá-las no IDLE ou inserir em um programa o comando **import** math, que ativa o módulo (*module*) math, e preceder cada função de math., p.ex. math.sqrt(4), math.e etc; os resultados são sempre do tipo float, a menos de observação em contrário. Para cálculos com números complexos, dar **import** cmath .

Função	Significado	Exemplos
atan(x)	Arcotangente, argumento e resultado em radianos	math.atan(2) → 1.1071487177940904;
ceil(x)	O menor inteiro >= x	math.ceil(4.7) → 5
cos(x)	Cosseno, x e resultado em radianos	math.cos(math.pi/2) → 6.123233995736766e-17 (devia ser zero; não é devido à aproximação); math.cos(math.pi) → -1.0
degrees(x)	Converte x em graus para radianos	math.degrees(math.pi) → 180.0
e	A constante e	math.e → 2.718281828459045
exp(x)	e**x	math.exp(1) → 2.718281828459045; math.exp(2) → 7.38905609893065

factorial(x)	Fatorial de x de tipo int, resultado int	math.factorial(5) → 120
floor(x)	Maior int <= x	math.floor(4.7) → 4
fsum	Somatória	Como a sum() da tabela Funções Nativas, mas arredondando
inf	A constante infinito (maior número em float representável)	math.inf → inf
log(x, base)	Logaritmo de x na base (opcional); sem base dá o log na base e	math.log(10) → 2.302585092994046; math.log(100,10) → 2.0
log10()	Logaritmo na base 10	math.log10(100) → 2.0; em geral mais precisa que math.log(x,10)
log2()	Logaritmo na base 2	math.log2(8) → 3.0
modf(x)	Dá a parte decimal e a inteira de x	math.modf(1.25) → (0.25, 1.0)
pi	O número pi	math.pi → 3.141592653589793 (ver exs. em sen, cos, tan)
radians(x)	Converte x em radianos para graus	math.radians(180) → 3.141592653589793
sin(x)	Seno, x e resultado em radianos	math.sin(math.pi/2) → 1.0; math.sin(math.pi) → 1.2246467991473532e-16 (devia ser zero; não é devido à aproximação)
sqrt()	Raiz quadrada	math.sqrt(4) → 2.0; math.sqrt(5.6) → 2.3664319132398464
tan(x)	Tangente, x e resultado em radianos	math.tan(math.pi) → 1.2246467991473532e-16 (devia ser zero); math.tan(math.pi/2) → 1.633123935319537e+16 (representação do infinito)
trunc(x)	Parte inteira de x	math.trunc(3.5) → 3

7. FUNÇÕES/MÉTODOS SOBRE LISTAS

Funções	Significado	Exemplos
append()	Adiciona um elemento ao final da lista	L=[1, 2, 3, 4]; L=L+[5] → [1, 2, 3, 4, 5]; L=L+'#' → [1, 2, 3, 4, 5, '#']
clear()	Limpa uma string ou um set	L = [1, 2, 'três']; L.clear(); print('d',x) → [] L = {1, 2, 'três'; L.clear(); print('c',x); → set();
copy()	Retorna uma cópia da lista como um novo objeto	L = [1, 2, 'três']; Lnova= L.copy(); print(Lnova) → [1, 2, 'três']
count(v)	Retorna o número de elementos com o valor v	L=[1, 2, 3, 2, 4]; Conta_L= L.count; print(Conta_L) → 2 L = ['a', 'b', 'c', 'b', 'b']; Conta_b = L.count('b') → 3 texto = "Olá mundo! Olá a todos!"; Conta_Olá = texto.count("Olá"); print(Conta_Olá) → 2
extend(s)	Adiciona os elementos da lista s (ou qualquer iterável, (V. 2.10), ao final da lista atual	L=[1,2,3,4]; L.extend([5,6]); print(L) →[1, 2, 3, 4, 5, 6]
index(e,c,f)	Retorna o índice da primeira ocorrência de e, começando no índice c e terminando no índice f	L = [1,2,'três']; i1 = L.index(1); i2 = L.index('três'); print(i1,i2) → 0 2 L = [1, 2, 3, 4, 1, 1, 1, 4, 5]; i1 = L.index(1, 4, 8); i2= L.index(1, 2, 8); print(i1,i2) &rar; 4 4
insert(p,e)	Adiciona um elemento e na posição p	L = [1, 2, 3, 4]; L.insert(2, 10); print(L) →[1, 2, 10, 3, 4]
lstrip()	Remove espaços em branco ou caracteres especificados no início de uma string	Como strip(), mas só do começo
pop(e,p)	Remove o elemento e na posição p; se p não for fornecido, remove o último elemento da lista	L = [1, 2, 3, 2, 4]; L.pop(3); print(L) → [1, 2, 3, 4]; print(L.pop()); → [1, 2, 3] pop() [pode ser usado para desempilhar o topo de uma pilha]
remove()	Remove o primeiro item com o valor especificado	L = [1, 2, 3, 2, 2]; L.remove(2); print(L) → [1, 3, 2, 2] L = ['a', 'b', 'c', 'b', 'b']; L.remove('b'); print(L) → ['a', 'c', 'b', 'b']
replace('a','b')	Substitui ocorrências de 'a' por 'b'	OLA = 'O?lá, b?o?m di?a!?!'; OLA_Limpo = OLA.replace('?', ''); print(OLA_Limpo) → Olá, bom dia!! [os duplos apóstrofes são essenciais]
reverse()	Inverte a ordem da listt	L = [1, 2, 3, 4]; L.reverse(); print(L) → [4, 3, 2, 1]
sort()	Ordena a lista	L = [1, 3, 4, 2]; L.sort(); print(L) → [1, 2, 3, 4]
s.split(sep)	Método da classe string. Gera uma lista com os elementos da string s separados pela string sep; se sep for omitido, usa branco como separador	s='a,b,3'; s.split(',') → ['a', 'b', '3']; s='x#y#z'; s.split('#') → ['x', 'y', 'z']
strip()	Remove espaços em branco ou caracteres especificados no início e no fim de uma cadeia	OLA = " Olá, bom dia!! "; OLA_Limpo = OLA.strip(); print(OLA_Limpo) → Olá, bom dia!! OLA = ",,##?? Olá, bom dia!! ,,##?? "; OLA_Limpo = OLA.strip(',#? '); print(OLA_Limpo) → Olá, bom dia!! [O espaço em branco no final da cadeia é essencial; a função interrompe a remoção de caracteres de ambas as extremidades quando encontra um caractere que não está nos caracteres especificados.]
rstrip()	Remove espaços em branco ou caracteres especificados no fim de uma string	Como a strip(), mas no fim da cadeia
translate()	Remove uma subcadeia usando None	OLA = 'O?la, b?o?m di?a!?!'; OLA_Limpo = OLA.translate({ord('?'): None}); print (OLA_Limpo) → Ola, bom dia!!

8. PRECEDÊNCIA (ORDEM DE EXECUÇÃO)

Ordem	Operador/função	Exemplos
1	(...)	(... ignorado pelo interpretador)
2	função()	abs(-5)+2 → 7
3	+ e - unários	-5-2 → -7; -(5-2) → -3
4	*, /, % e //	
5	+ e - binários (soma e subtração)	

6	& ("e" bit a bit)	
7	e ^	
8	<=, <, >, >=	
9	=, %=, /=, //= e -=	
10	+=, *= e **=	
11	in, not in	
12	not, or, and	

9. DECLARAÇÃO E USO DE UMA FUNÇÃO

Na sequência de um programa, a declaração de uma função deve sempre vir *antes* de seu uso. Uma função f pode ser declarada com *parâmetros*, dentro dos parênteses seguintes ao nome, como em **def** f(x,y). Naativação de uma função f, popularmente denominada de chamada" da função (*function call*), são "passados" *argumentos* que serão usados no lugar dos parâmetros, pela ordem, p.ex. f(10,"abc"); na execução de f, x assume o valor 10 e y o valor 'abc'. Os exemplos seguintes usaram o interpretador IDLE, daí o *prompt* >>>.

Sintaxe	Exemplos no IDLE
# Declaração (atenção para o alinhamento vertical) def nome-da-função (parâmetro-1, ..., parâmetro-m): comando-1 ... comando-n próximo-comando # Uso da função nome-da-função (argumento-1, ..., argumento-m) No IDLE, é necessário dar uma linha em branco para encerrar a declaração de uma função (sobre o IDLE, ver o item 16 abaixo).	>>> def soma(a,b): # declaração ... return a+b ... #Produzido pelo IDLE ao se dar um Enter ... >>> soma(1,2) # ativação 3 >>> soma (A*3,D) 4.2

10. NOTAÇÃO LAMBDA

Essa notação permite que se declare uma função sem dar-lhe um nome, colocando-a em qualquer lugar em que uma função possa ser ativada.

Sintaxe	Exemplos no IDLE
lambda lista-de-parâmetros: expressão com esses parâmetros	>>> y = lambda x: x**2 >>> y(8) 64 >>> min = lambda x,y: x if x < y else y >>> min (3,2) 2 >>> itens = [1, 2, 3, 4, 5]; >>> list(map(lambda x: x**2, itens)) → [1, 4, 9, 16, 25] >>> number_list = range(-5, 10) >>> list(fílder(lambda x: x < 0, number_list)) → [-5, -4, -3, -2, -1]

11. GLOBAL AND LOCAL IDENTIFIERS

Um identificador declarado dentro de uma função é somente local a ela (válido dentro dela); declarado fora da função, antes ou depois dela, em um escopo (isto é, espaço de validade) englobando diretamente a função, ele é global, e pode ser usado tanto fora como dentro da função. O uso de um identificador local evita muitos erros, pois só a função onde ele está declarado pode modificar seu valor; esse identificador fica "encapsulado" na função. Nesse sentido, o correto é passar valores para a função e obtê-los por meio de parâmetros na sua declaração (argumentos na sua ativação).

<pre>>>> def F(): ... Loc = 1 # local a F ... print (Loc, Glob) >>> Glob = 2 # global a F >>> F() 1 2 >>> Loc Loc NameError: name 'Loc' is not defined</pre>	Para converter um identificador local em global: <pre>>>> def F(): ... global Glob ... print (Glob) >>> Glob=1 >>> F() 1</pre>
--	---

Uma função F2 pode ser declarada dentro de uma outra função F1. Nesse caso, F2 torna-se local a F1 e não pode ser ativada fora de F1:

<pre>>>> def F1(): ... def F2(): # local a F1! ... LocF2 = 13 ... print (LocF2) >>> F1() → F2() NameError: name 'F2' is not defined</pre>	<pre>>>> def F1(): ... def F2(): ... LocF2 = 13 ... print (LocF2) ... F2() → # ativada dentro de F1 >>> F1() 13</pre>
---	---

Se a função F2 estiver declarada dentro de F1, a declaração **nonlocal** faz com que uma variável V declarada em F2 passe a ter o escopo de F1, mas não é válida fora de F1. V tem que ter um valor atribuído a ela em F1 *antes* da declaração **nonlocal**:

<pre>>>> def F1(): ... LocF1 = 13 # Local a F1 necessária! ... print ("Na F1:", LocF1) ... def F2(): ... nonlocal LocF1 # Global a F2 ... LocF1 = "26"</pre>	<pre>>>> def F1(): ... def F2(): ... Local = 1 ... print(Local) >>> F1()</pre>
--	--

<pre> print ("Passou pela F2:", LocF1) F2() # Na F2 >>> F1() Na F1: 13 Passou pela F2: 26 </pre>	<pre> >>> F2 NameError: name 'F2' is not defined </pre>
---	--

12. CLASSES

12.1 Introdução

Classes podem ser conceitualmente encaradas como uma extensão dos tipos e de funções, e são usadas para se obter mais encapsulamento. Ao contrário das funções, as classes não contêm parâmetros na declaração de seu nome; para serem usados com argumentos na ativação ("chamada") da classe, eles devem ser declarados no corpo da classe com um primeiro atributo; em geral é costume chamá-lo de `self` (ver exemplo abaixo), mas pode ter um nome qualquer. A função nativa `__init__()` declara as propriedades iniciais da classe e é ativada toda vez que é criado um objeto da classe (ver abaixo). Em seu corpo, as classes podem conter declarações de variáveis e de funções (estas, denominadas de *métodos* da classe), que se tornam locais a elas. Cada elemento declarado em uma classe é denominado "atributo" ou (para englobar funções) *propriedade* da classe, e é referenciado pelo nome da classe, um ponto e o nome da propriedade. Como as funções, as classes devem ser definidas antes de serem usadas. Podem-se atribuir valores aos elementos de uma classe fora dela. Uma classe pode ser atribuída a uma variável `V`; nesse caso `V` recebe uma *instância* da classe, um *objeto* com todas as propriedades da classe, podendo-se produzir quantas instâncias de uma classe que se quiser. É possível alterar o valor de um atributo de um objeto (ver exemplo abaixo). Uma classe `C2` pode ser declarada como sendo subordinada a outra classe `C1`; `C2` é denominada *subclasse* de `C1`, e `C1` uma *superclasse* de `C2`. Nesse caso, `C2` "herda" todas as propriedades de `C1`, mas pode ter suas próprias propriedades locais, que não são acessíveis por `C1`. Uma classe pode ser criada dinamicamente usando a função `type(N,B,dic)`, onde `N` será o nome da classe, `B` uma n-pla que serve de base, da qual a classe senta criada herda os atributos (isto é, `N` será uma subclasse de `B`), e `dic` é um dicionário contendo os nomes dos atributos e valores que `N` vai conter. Para eliminar um objeto já criado `p`, usar **del** `p`. Ver outras funções que se aplicam a classes no item 5 acima.

<pre> class C: """Esta é uma classe.""" # (Atributo implícito: __doc__, para documentação X = 13 def FdeC (Y): return Y + 1 print(C.X) #Acesso a um dos atributos de C → 13 print(C.FdeC(2)) #Acesso à função de C → 3 print(C.__doc__) #Documentação no cabeçalho de C → Esta é uma classe ObjC = C() #Instanciação: criação de um objeto print(ObjC.X) 13 </pre>	Criação dinâmica de uma nova classe <pre> C2 = type('C2', (object,), {'atr': 100}) inst = C2() # Criação de um objeto, uma instância de C2 inst.atr # Acesso a um dos atributos da instância 100 </pre> Se <code>C2</code> for declarada como uma subclasse de uma classe <code>C1</code>, usar <code>C1</code> em lugar de <code>object</code>; nesse caso, <code>C2</code> herda todos os atributos de <code>C2</code>
<pre> class pessoa: def __init__(self, nome, idade) #Parâmetros nome, idade self.N = nome #self em lugar do nome da classe self.I = idade p1 = pessoa('Val',84) #declaração do objeto p1, uso de argumentos print(p1.N, p1.I) → Val 84 </pre>	<pre> class Person: Age = 84 p1 = Person # Criação do objeto p1 print(p1.Age) p1.Age = 48 # Modificação do valor do atributo do objeto print(p1.Age)→ 84 48 </pre>

12.2 Polimorfismo

Polimorfismo é a capacidade de um mesmo operador, função, ou objeto de uma classe operar com argumentos de tipos diferentes. Por exemplo, o operador `+` pode indicar soma de inteiros, concatenação de cadeias ou de n-plas. etc. A função `len(x)` aceita, como argumento `x`, cadeias (dando o número de toques), ou uma lista, n-pla ou conjunto (dando o número de elementos). Tudo isso é pré-definido. O polimorfismo de funções ou métodos de classes é algo definido pelo programador. Uma mesma função `f` pode ocorrer em classes diferentes. Nesse caso, `f` pode ser aplicada a várias instâncias (objetos) dessas classes, como exemplificado abaixo. Isso ajuda a implementar e usar uma "programoteca" pessoal – já existem as de módulos disponíveis e a de muitos colaboradores, permitindo a re-utilização de partes de um programa.

<pre> class Azul: def cor(self): return "Azul" class Verde: def cor(self): #polimorfismo do método cor return "Verde" p1 = Azul() # objetos - instanciações das classes p2 = Verde() for i in (p1, p2): #i assume um objeto print(i.cor()) → Azul Verde </pre>	<pre> class Felino: def __init__(self, animal): self.F=animal def som(self): return self.F, "faz miau" class Canino: def __init__(self, animal): self.C = animal def som(self): #polimorfismo do método som return self.C, "faz au au" for i in (Felino('gato'), Canino('cão')): #i assume o objeto de uma classe print(i.som()) → ('gato', 'faz miau') ('cão', 'faz au au') </pre>
--	---

12.3 Herança

O conceito de herança é bastante antigo nas linguagens de programação. Ele surgiu com a linguagem Algol, que teve sua definição final em 1960, com a Agol-60. Nessa linguagem, um bloco era declarado com comandos entre `begin ... end`. Era possível declarar um bloco `B2` dentro de um bloco `B1`. Nesse caso, todas as variáveis e procedimentos (denominados "funções" na posterior linguagem C) declaradas em `B1` eram acessíveis em `B2` (denominadas de *globais* a `B2`), mas as declaradas em `B2` não eram acessíveis dentro de `B1` (eram *locais* a `B2`). Convencionou-se dizer que `B2` *herdava* as propriedades delcaradas em `B1`. Em Algol, uma função especial era um tipo de procedimento que retornava um valor, como uma função em Python. Na linguagem C não era possível declarar uma função dentro de outra: todas as funções tinham um nível léxico só (o mais externo), o que foi imposto pelas limitações da máquina (PDP-11) para a qual a linguagem foi desenvolvida. Isto é, em lugar de se mudar a estrutura da máquina, entortou-se a programação. Python permite que se declare uma função `F2` dentro de outra `F1`, o que é denominado de *funções encaixadas* (*nested fuctions*); dentro de `F2` estão disponíveis todas as variáveis e funções ("propriedades") declaradas em `F1`. Estabeleceu-se a denominação de que há uma *herança* (*inheritance*) de `F2` em relação a `F1`, isto é, `F2` *herda* os atributos e funções declarados em `F1`, que são *globais* a `F2`, e os declarados em `F2`, que não estão disponíveis em `F2` fora de `F1`, são *locais* a `F1`. As propriedades de `F1` têm um escopo (*scope*) que abrange toda `F2`; o escopo das propriedades de `F2` têm apenas o escopo da `F2` (ver esquema abaixo).

A noção de herança também se aplica às classes: é possível declarar uma classe `C2` como sendo uma *subclasse* de outra classe `C1`. A linguagem C++ introduziu em C a noção de classes, e como em C não havia declaração de função dentro de função, essa sintaxe passou para a C++ em relação às classes. Python preservou essa estrutura. Para que `C2` herde todas as propriedades de `C1` é necessário declará-las como no esquema seguinte:

<pre> def F1(): # propriedades próprias de F1; as de F2 não estão disponíveis </pre>	<pre> class C1: # Superclasse de C2 # propriedades próprias de C1; as de C2 não estão disponíveis </pre>
--	--

def F2(): # propriedades próprias de F2; as de F1 estão disponíveis	class C2(C1): # Subclasse de C1 # propriedades próprias de C2; as de C1 estão disponíveis
---	---

A noção de herança é muito útil, pois no exemplo pode-se declarar a classe C2 sem que seus atributos próprios sejam alterados fora dela. Isso facilita o encapsulamento, isto é, isolamento em relação a influências externas. [Ver exemplos.](#)

Finalmente, uma observação filosófica: na noção de herança tem-se mais uma denominação indevida na computação, que está cheia de antropomorfismos enganosos: minerais e vegetais não herdam; apenas animais e seres humanos herdam. As máquinas estão em um nível inferior aos minerais.

13. COMANDOS SIMPLES E COMPOSTOS

Syntax	Examples (testados no Azure, V. Ambientes abaixo)
13.1 Bloco (de comandos): Comando; Comando; ... ; Comando #todos na mesma linha ou todos alinhados verticalmente à esquerda, ou em alguma coluna, se o bloco estiver imerso em algum comando: Comando Comando ... Comando	<pre>J = 2; K = 3; M = 4; J,K,M (2,3,4) J=2 K=3 M=4 J,K,M (2,3,4)</pre>
13.2 if – comando de escolha lógica # atenção para o alinhamento vertical if Expressão Lógica: Bloco #qualquer coluna a partir da 2a. # em relação ao if elif Expressão Lógica: #opcional, # alinhado na mesma coluna que o if Bloco #a partir de qualquer coluna elif Expressão Lógica: # Idem Bloco # Idem ... elif Expressão Lógica: # Idem, opcional Bloco # A partir de qualquer coluna else: ""Opcional. No IDLE, o else deve começar na 1a. coluna; segue-se um só Comando ou um Bloco com várias linhas começando na próxima a partir da 2a coluna em relação ao else "" Próximo-Comando #alinhado na coluna do if	<pre>J = 2; K = 3; L = 4 # válido para todos os exemplos seguintes if J < K: print(J); print(K)→ 2 3 if K>J : "" O ":" pode estar em qualquer coluna desta linha "" N = J+3 P = K+4 N, P → (5,7) {PC} corrigiu o (5, 7) if K < J: N=5 else: N=6 P=7 N, P → (6,7)</pre>

13.3 while – comando de repetição da execução (malha de execução, <i>loop</i>) de um bloco de comandos while Expressão Lógica: Comando; ""Opcional: um só comando! Ou Bloco (comandos alinhados verticalmente à direita do w) else: Bloco #Opcional, numa só linha ou Bloco próximo comando O comando else é executado quando a Expressão Lógica der valor False . O comando break interrompe a execução da malha de repetição e desvia para o próximo comando após o else com seu bloco. É conveniente usá-lo quando ocorre uma situação de exceção durante a execução da malha. Dentro do bloco de comandos pode ocorrer um if Expressão Lógica: Continue Se Expressão Lógica for verdadeira, pula o restante da presente iteração do while continuando com a próxima iteração. Aparentemente, o continue não pode ocorrer logo depois do while .	Atenção: ao usar o comando while no IDLE, ele é executado até o fim (até Epressão Lógica ficar com valor False) antes de se poder dar o próximo comando. L = 4
<pre>M = 1 while M<L: M += 1; print(M) → 4 M = 1 while M < 4: print(M) M += 1 → 1 2 3 print(10) → 10</pre>	<pre>M = 1 while M < 4: print(M) M += 2 else: print(7); print(8) → 1 3 7 8 print(10) → 10 i = 0; L = [] while i < 3: i += 1 if i == 2: continue L = L + [i] print(L) → [1, 3]</pre>

13.4 for – comando de malha de repetição for Variável in Lista de Expressões: Bloco else: #Opcional Bloco Os valores da Lista de Expressões são atribuídos a Variável; para cada atribuição o Bloco do for é executado uma vez. Quando a lista é esgotada, é executado o bloco do else , se este existir. O comando break interrompe a execução da malha de repetição e vai para o próximo comando depois do for .	<pre>for I in range(3): #começa em 0! print(I) → 0 1 2 for I in range(2,4): # começa em I # e termina em 3 (4-1)! → 2 3 Frutas = ['caju','caqui','manga'] for fruta in Frutas: print ('Fruta da vez:', fruta) → Fruta da vez: caju Fruta da vez: caqui Fruta da vez: manga</pre>	<pre>for letra in 'xyz': print ('Letra da vez:', letter) → Letra da vez: x Letra da vez: y Letra da vez: z Uso do else e dois fors encaixados for I in range(3): for J in [2,'xy']: # uso do else print ('I =", I, 'J =", J] else: print ('Passou por aqui!') → I = 0 J = 2 I = 0 J = xy Passou por aqui! I = 1 J = 2 I = 1 J = xy Passou por aqui! I = 2 J = 2 I = 2 J = xy</pre>
--	--	--

O comando continue visto no while acima pode ser também usado em uma malha do for .	<pre>for I in range(5): if I==2: break print(I) → 0 1</pre>	
Uso do for em varredura de iteráveis (V. 2.10)		
<pre>for i in [1, 2, 3]: print (i) → 1 2 3</pre>	<pre>for car in "123": print (car) → 1 2 3</pre>	<pre>for indice in {'um':1, 'dois':2}: print (indice) → dois um</pre>

13.5 match...case – comandode escolhas múltiplas	
match Expressão: case Expressão1: código1 a executar se Expressão tem o mesmo valor que a Expressão1 case Expressão2: código2 a executar se Expressão tem mesmo valor que a Expressão2 ... case ExpressãoN: códigoN a executar se Expressão tem mesmo valor que a ExpressãoN case _: código a executar se Expressão1 ... ExpressãoN têm todas valores diferentes de Expressão; esse último case é opcional	Suponhamos que a variável Cor tenha um valor 'azul', ou verde', ou 'roxo', ou nenhum desses, e que se queira exibir o valor : match Cor: case 'azul' : Print ('A cor é azul') case 'verde': Print ('A cor é verde') case 'roxo' : Print ('A cor é roxa') case _ : Print (A cor não é nem azul, nem verde, nem roxo') O comando match substitui com muitas vantagens um encadeamento de if's quando se testa somente o valor de certa expressão: if <Expressão> = ... : elif <Expressão> = ... : elif <Expressão> = ... : ... elif <Expressão> = ... : else ...

13.6 import – inclusão de um módulo	
Este comando inclui um módulo em um programa, ou somente algumas de suas funcionalidades. P.ex., para usar uma função matemática como sqrt (raiz quadrada), é necessário importar o módulo math. Mas se o programa usa apenas a função sqrt, ela pode ser importada isoladamente usando o comando from. from módulo import função Para usar um outro nome de um módulo (<i>alias</i>): import módulo as novo nome	import math; math.sqrt (4) → 2 from math import pi; pi → 3.141592653589793 import math; math.e → 2.718281828459045
13.5 try – comando de detecção de exceção (erro)	
try: comando1 except: comando2 except: comando3 else: comando4 finally: comando5 Os comandos 1 a 5 podem ser blocos de comandos. O comando try verifica se algo em comando1 é indefinido ou produz um erro, de modo que comando1 não pode ser executado. Se assim for, o comando2 e o comando3 são executados (pode haver um só ou mais comandos except). O comando4 do else (opcional) é executado se não houve algum erro no comando1. O comando5 do finally (opcional) é executado independentemente de ter havido um erro no comando1. Como o nome try diz, ele serve também para se testar se haverá um erro em comando1, antes que o erro ocorra e a execução pare. Pode-se encaixar um try dentro de outro try	try: print (x) except: print ("Ocorreu um erro, provavelmente x não foi declarada") Suponha a existência de uma lista de nome L. O comando try pode ser usado para verificar se o elemento L[j] não está na lista L, caso em que o valor de j é maior do que o índice do último elemento de L, ou negativo, cf. o item 2.5 acima: try: A = L[j] except: print ("Erro: L[, j, "] não está na lista")
13.8 pass – ignora um comando	
Esse comando, inserido dentro de um outro composto, faz com que o sistema ignore esse outro. Ele é muito prático durante a elaboração de um programa. Ver o exemplo. Veja muitos exemplos do pass.	def UmaFuncao(x): pass O corpo de UmaFuncao ainda não foi programado, mas já se sabe que ele deverá sê-lo e a função ficará nesse trecho do programa. Com isso garante-se que o programa todo será executado; sem o pass ele não o seria, haveria um erro de sintaxe. No 2º exemplo do try acima, se nada deve ser feito se o L[j] não estiver na lista, pode-se usar try: L[j] except: pass
13.9 raise – força uma exceção	
Esse comando força a ocorrência de um erro, e encerra a execução Veja descrição e exemplos do raise. Os exemplos à direita foram copiados dessa página.	a = 5 if a % 2 != 0: raise Exception("O número não pode ser um inteiro ímpar") → Traceback (most recent call last): File caminho do programa, line 4, in módulo

raise SystemExit("Mensagem") Produz o término da execução do programa, exibindo a Mensagem	<pre>raise Exception("O número não pode ser um inteiro ímpar") → Exception: O número não pode ser um inteiro ímpar</pre> No exemplo abaixo, testa-se se uma conversão de tipo é válida; ValueError é uma classe que produz a impressão de seu nome e do argumento: <pre>s = 'jabuticaba' try: num = int(s) except: ValueError: raise ValueError("Cadeia", s, "não pode ser convertida para inteiro")</pre> No resultado, além das linhas do exemplo acima e mensagens, aparece: <pre>raise ValueError("Cadeia não pode ser convertida para inteiro")</pre>
13.10 yield x – força x como resultado de uma função Este comando deve ser usado dentro de uma função, gerando um resultado para a mesma. Ele não interrompe a execução da função, ao contrário de return , que retorna um só valor e encerra a execução da função. O yield pode ser colocado em vários locais dentro de uma função, e gerará o valor da função toda vez que for executado. Já o return também pode ocorrer em vários locais, mas se for executado encerra a execução da função. O exemplo ao lado foi tirado daqui .	<pre>def ParOuImpar(x): for i in range(x): if (i % 2 == 0): yield 'par' else: yield 'ímpar' y=ParOuImpar(5) for j in y: print(j) → par ímpar par ímpar par</pre>

14. MÓDULOS

14.1 Introdução

"Módulo" é uma denominação dada em Python para um programa M que pode ser inserido em um programa P. Para isso é necessário que P "importe" M, usando no primeiro o comando **import** M. Na importação, é possível mudar localmente o nome de um módulo M para MM com **import M as MM**. Para se criar um módulo M, deve-se criar o programa de M e depois armazená-lo com algum nome, por exemplo M, e a extensão necessária .py, isto é, M.py. Dessa maneira, é possível criar e usar bibliotecas de programas ("programotecas"), reutilizando programas já testados. Um módulo M pode conter suas próprias variáveis e funções, que são usados como se M fosse uma classe. Se uma variável V e uma função f são declaradas dentro de M, para usá-las no programa P emprega-se M.V e M.f().

14.2 Alguns módulos

Abaixo são citados alguns módulos bem úteis. Para usá-los, é necessário ver a documentação correspondente, que pode ser encontrada usando-se Python e o nome do módulo.

1. numpy é um módulo que permite computação científica e processamento de matrizes, números aleatórios, transformada de Fourier (análise harmônica) etc.
2. random é usado para geração de números aleatórios
3. math - ver a seção 6 acima.
4. PyTest facilita o teste de programas.
5. Pyperclip permite inserir texto na área de descarte (clipboard).
6. Matplotlib é um módulo básico para gerar imagens de gráficos.
7. Dash é um módulo que permite criarem-se imagens de vários tipos, como um relógio, gráficos lineares e de pizza etc.
8. Seaborn permite desenhar gráficos estatísticos.
9. PyGame possibilita o uso de rotinas multimídia para áudio, teclado, mouse, etc.
10. Pillow é uma parte da Python Image Library, (módulo Image) permitindo criar *thumbnails*, aplicar filtros, exibir imagens etc.
11. Requests é um módulo que permite acesso à internet e uso de http, incluindo autenticação, uso de cookies, trabalhar com sessões, proxies etc. Permite usar o método Jason evitando adicionar links da internet manualmente.
12. Chardet é um módulo que contém rotinas para detecção do tipo de caracteres usados em um texto.
13. Progress é um módulo que permite gerar um gráfico de barras do progresso de um procedimento.

15. PALAVRAS RESERVADAS

As seguintes palavras não podem ser usadas como nomes de identificadores (variáveis e funções):

```
assert and as break class
continue def del elif else
except False finally for from
global if import in lambda
match None nonlocal not or pass
is raise return True try while
with yield
```

16. REFERÊNCIAS

16.1 Tutoriais

- Tutorial livre: www.educba.com/category/software-development/software-development-tutorials/python-tutorial/
- Tutorial em português da UFF (Python versão 2!): www.telecom.uff.br/pet/petws/downloads/tutoriais/python/tut_python_2k100127.pdf
- Tutorial "oficial": <https://docs.python.org/3/tutorial>
- Tutoriais sobre operadores: www.programiz.com/python-programming/operators; https://www.tutorialspoint.com/python/python_basic_operators.htm
- Tutorial sobre variáveis, constantes e tipos: www.tutorialspoint.com/python/python_variable_types.htm
- Tutorial sobre [uso de listas](#) com muitos exemplos e testes de conhecimento
- Tutorial sobre o uso de matrizes (*arrays*): www.i-programmer.info/programming/python/3942-arrays-in-python.html
- Tutorial sobre o uso do módulo NumPy para uso de matrizes: www.i-programmer.info/programming/python/5785-advanced-python-arrays-introducing-numpy.html?start=1
- Tutorial sobre classes: <https://docs.python.org/3/tutorial/classes.html>

- Tutorial sobre módulos: <https://docs.python.org/3/tutorial/modules.html>

16.2 Outras referências

- <https://docs.python.org/3/library/functions.html> (a tabela de funções nativas foi tirada da versão 2)
- <https://ddi.ifi.lmu.de/probestudium/2012/ws-i-3d-programmierung/tutorials/python-referenzkarte> (em inglês); essas folhas de consulta são chamadas de *Cheat Sheet* (cola); melhor seria *reference sheet*.
- *Cheat sheet* dirigida a administradores de redes, com lista de *libraries*, métodos e módulos, produzidas por PCWDSL: www.pcwold.com/python-cheat-sheet
- https://s3.amazonaws.com/assets.datacamp.com/blog_assets/PythonForDataScience.pdf
- Tipos: <https://docs.python.org/3/library/stdtypes.html>
- Funções matemáticas: <https://docs.python.org/2/library/math.html>
- Funções com números complexos: <https://docs.python.org/3/library/cmath.html#module-cmath>
- Precedências: www.tutorialspoint.com/python/operators_precedence_example.htm
- Manual de referência: <https://docs.python.org/3/>
- Site "oficial": <https://www.python.org/>
- Grupo de discussão de Python no Brasil: <https://python.org.br/>
- Livro: Nilo Ney Coutinho Menezes, *Introdução à Programação com Python: Algoritmos e lógica de programação para iniciantes*, 2ªed, Novate, 2018

17. INSTALAÇÃO DO PYTHON E USO DO INTERPRETADOR IDLE E SEU EDITOR

17.1 Instalação

Para instalar o Python e usar o seu interpretador IDLE (*Integrated Develop and Learning*): <https://www.python.org/downloads>. O IDLE permite digitarem-se programas; cada linha com um comando digitada no IDLE é executada imediatamente ao se encerrá-la com Enter. Experimente, por exemplo, com a sequência A = 5 Enter A Enter. O valor 5 de A será exibido.

17.2 Uso do IDLE no Windows

Em meu W11, em 8/7/24 o Python 3.10.5 executável foi instalado automaticamente com o Python 3.12.4 no arquivo c:\Windows\py.exe. Para descobrir o caminho desse diretório, acione com o botão direito o ícone do Python se ele estiver na área de trabalho, ou na lista de programas preferidos do menu Iniciar, em seguida Propriedades, aba Atalho; o caminho está no campo Destino e pode ser copiado. No meu caso, para acionar o IDLE basta fazer uma busca do Windows com py., pois o caminho deve ter sido adicionado ao Path do Windows. **2.** Acionando o programa Python, aparece uma janela do IDLE, parecida com uma janela de *prompt* do Windows, aqui chamada "janela de comandos". Depois da carga aparece o *prompt* do Python, >>> e se podem digitar comandos da linguagem. Várias janelas de comandos podem ser abertas, ativando-se a Python mais de uma vez como descrito, permitindo copiar e colar de uma para outra. **3.** A edição de um comando na janela de comandos do Python segue o padrão do Windows. **4.** É necessário digitar uma linha de cada vez, terminada com Enter. Note-se que ao terminar de digitar um comando, dando-se Enter o IDLE executa o comando e abre uma nova linha com >>>. Se for um comando composto ou declaração de função, aparece ... na próxima linha, onde deve ser digitada a continuação. Para encerrar um comando composto ou declaração de função, é necessário dar linhas em branco, o que provoca a execução do comando composto, ou a função fica declarada, podendo ser usada posteriormente, aparecendo o >>> novamente. Além disso, pode-se da Ctrl C, que interrompe a espera da digitação, sem que se perca o estado do interpretador, como valor das variáveis; no entanto, cancela a digitação de um comando composto. **6.** Para carregar um programa em Python (extensão .py), use File → Open. **5.** Para fechar a janela de comandos, dê quit() ou, no início da próxima linha de comando, Ctrl+D. Estando em uma janela do IDLE, para eliminar todas as variáveis e identificadores criados com a última sessão/programa, e continuar com um novo programa, use no menu principal Shell &raarr; Restart Shell. Aparece uma linha indicando esse Restart. **7.** É possível ativar o IDLE diretamente de uma janela normal de *prompt* do Windows. Para isso, nessa janela desvie (com o comando > cd nome do diretório) para o diretório onde está o executável da Python e dê o comando py. Se o py tiver sido inserido no PATH do Windows no *download* da Python, basta dar py em qualquer linha de comando numa janela de Prompt. A desvantagem de usar o Prompt do Windows é de se ter apenas as suas possibilidades arrevesadas de edição. Em uma janela do IDLE tem-se mais possibilidades, por exemplo ao digitar um comando **if** (se o : no fim da condição for esquecido, uma mensagem apontará isso) a próxima linha com **then** aparecerá já com tabulação. **8.** Para uma lista dos nomes das variáveis ativas no IDLE, use dir(). Para eliminar uma variável x (não estará mais disponível), use del (x).

17.3 O Editor do IDLE e execução de um programa armazenado

O IDLE tem um Editor que é ativado em uma janela separada. Atenção: prefira usar o Editor para testar programas em lugar o IDLE. É mais simples e prático. Para ativá-lo, no menu do IDLE acione File → New File. É aberta uma janela em branco do Editor, onde se pode digitar um programa, tendo-se as facilidades de edição do IDLE e as do Windows. Ao se completar uma linha, o Enter obviamente não produz sua execução, ao contrário do IDLE. **2.** Terminada a digitação do programa, é necessário armazená-lo usando no menu do editor File → Save. Aparece o nome do arquivo na primeira linha, entre asteriscos, indicando que ele foi armazenado e o seu caminho. **3.** Se um programa está armazenado em um arquivo, no Editor pode-se carregá-lo acionando no seu menu File → Open (isso pode ser feito também na janela do IDLE, que abre uma do Editor), sendo que o nome do arquivo deve ter a extensão .py. **4.** Em seguida, o programa que está no editor pode ser executado, ativando-se no menu do Editor: File → Save e depois Run → Run Module. Isso limpa tudo o que estava no no IDLE (variáveis, que deixam de existir, estado do programa anterior etc.), exibe nele a mensagem === RESTART: caminho do arquivo com o programa que foi executado ===, copia o programa do Editor para o IDLE, e o executa. **5.** Atenção: se no programa executado a variável A está com um valor, p.ex. 5, se numa linha do editor colocar-se A, ao executar o programa com Run Module não vai aparecer o valor de A logo na linha seguinte, como ocorreria se o programa estivesse na janela do IDLE. Isso é óbvio, pois inserindo-se um comando em uma linha do IDLE e dando Enter o comando é interpretado e executado. Ao se executar um programa completo copiado da janela do Editor, não há essa interpretação comando a comando, dando seu resultado. Para se exibir o valor de A ao executar um programa que está no Editor é necessário inserir o comando print(A) ou print("A=", A). **6.** Para editar um programa fora do IDLE e do Editor, use o Bloco de Notas do Windows (ou copie o programa que foi editado em outro editor, para uma janela do Bloco de Notas). Ao armazenar o texto, para ser carregado no Editor, no campo Tipo escolha Todos os arquivos, em Codificação escolha UTF-8, coloque no nome a extensão .py e salve o programa. Uma outra possibilidade é salvar como .txt e depois mudar a extensão para .py. Para isso, use por exemplo o programa Total Commander, que eu uso em lugar do Windows Explorer, pois é muito mais prático, já que apresenta duas partes em uma janela, cada uma com um diretório, podendo-se mover ou copiar de uma parte para a outra, podem-se fazer buscas por nomes de arquivos ou diretórios, mudar extensões dos nomes etc.

17.4 Uso do depurador do IDLE (Debugger)

Para detectar erros em um programa armazenado localmente, pode-se usar o Debugger do IDLE. Ver um excelente tutorial dele em

www.cs.uky.edu/~keen/help/debug-tutorial/debug.html

1. Carregue um programa no Editor (ver item 3 acima). **2.** Insira *breakpoints*, pontos de parada, em lugares onde se quer que a execução pare, por exemplo para se verificar o valor de variáveis ou a sequência de execução de comandos. Para isso, no Editor acione o botão direito do *mouse* na linha onde quer inserir um ponto de parada, e acione Set Breakpoint. Aparece uma tarja amarela na linha. Para se eliminar um ponto de parada, idem Clear Breakpoint. **3.** No Editor, acione Window → Show Line Numbers; aparecem os números das linhas do programa, importante para se acompanhar o Debugger. **4.** No IDLE, acione Debug → Debugger; aparece a mensagem DEBUG ON. Repetindo, desliga-se a execução do Debugger e aparece DEBUG OFF no IDLE. **5.** No Editor, acione Run → Run Module; aparece uma janela Debug Control. Nela, abaixo dos *buttons* aparece o nome do programa e na área de trabalho uma linha com tarja azul com > e no fim dela o número da linha onde foi inserido o ponto de parada onde a execução parou. **6.** Acionando o *button* Step o programa será executado linha a linha, sendo a atual trajada de cinza, *antes* de ser executada. Se a linha contiver uma atribuição de um valor a uma variável, depois da execução dessa linha essa variável é exibida na janela do Debugger e o valor que ela recebeu aparece na área Locals, juntamente com identificadores e valores de todas as variáveis que já receberam valor (assim pode-se conferir o que ocorrerá com a condição de comandos como o **if** e o **while**). Esse comando Step é talvez o mais útil, pois permite acompanhar toda a execução. **7.** Acionando-se o *button* Go o programa é executado sem parar até o próximo ponto de parada. Assim, se houver dúvida sobre a execução de um trecho do programa, pode-se delimitá-lo com dois pontos de parada, desviar inicialmente para o primeiro com o Go, e daí em diante usar o Step; ao se atingir o segundo ponto, pode-se dar novamente o Go para ir para o ponto inicial da parada. **8.** Se houver a execução de uma função input(), o processo vai parar. É necessário ir à janela do IDLE e dar o valor solicitado e Enter. O valor dado, com a variável que recebeu a entrada, aparecem na seção Locals. **9.** Se ocorrer um erro durante a execução do programa, a mensagem aparecerá na janela do Debugger. **10.** O button Quit deve encerrar a execução do debugger. **11.** Ainda não descobri para que serve o *button* Out, talvez para desviar para o janela do Editor.

18. CURSO

- Curso de Python em português no Coursera (grátis sem certificado), por Fábio Kon (IME-USP): <https://www.coursera.org/learn/ciencia-computacao-python-conceitos>

Atenção: nesse curso o Prof. Kon dá exemplos nos quais ele edita um programa em um editor de textos, depois armazena-o por exemplo no arquivo de nome programa.py . Em seguida, ele ativa o interpretador da Python usando o comando ou algo parecido `...> python35 programa.py` . Ocorre que o IDLE usa a codificação de caracteres UTF-8, de modo que o texto do programa deve ser armazenado localmente nessa codificação, que existe em alguns editores de texto. No Windows, que normalmente usa a codificação Latin-1 (não aceita pelo IDLE), o Bloco de Notas tem essa opção, na opção Salvar como → Codificação e selecionando UTF-8. Idem para os programas que o curso exige serem enviados para o Coursera para serem executados e avaliados nesse sistema. Se isso não for feito, ao executar o programa aparece uma mensagem de erro de caractere não reconhecido.

- Cursos grátis em inglês: www.freecodecamp.org/news/learn-python-free-python-courses-for-beginners

19. TEXTOS, AMBIENTES DE PROGRAMAÇÃO, DOCUMENTAÇÃO E FÓRUNS DE PROGRAMAÇÃO

- [Livro de Luciano Ramalho](#)
- [Spyder](#): ambiente de programação e depuração (IDE), que me foi fortemente recomendado por {AL} como sendo muito melhor do que o IDLE e o seu Editor
- [Jupyter](#): sistema na internet que, usando um navegador, serve como um ambiente de programação e depuração com recursos para programação literária (código fonte e documentação em um único arquivo). Funciona tanto no computador local, quanto em um remoto na Internet. Não é preciso instalar o IDLE. Para um método de documentação inserida nos programas, com vários níveis de abstração, veja o [artigo](#).
- [Azure](#): ambiente da Microsoft; inclui o Jupyter e provê recursos adicionais. Programas ficam em notebooks. Para criar um, entre em sua conta, acione Notebooks (no menu à esquerda) e depois + New (no canto inferior esquerdo). Em cada célula de um notebook é possível colocar um programa e modificá-lo; para executá-lo, selecione-o e dê Shift Enter. Uma variável ou função declarada em uma célula já executada é válida nas outras células. Aparentemente, não inclui o depurador (*debugger*) do IDL.
- [PyCharm](#), um ambiente de programação com versão *open source* (sem custo). {LF}
- [Anaconda](#), um ambiente *open source* de programação incluindo sistemas para *machine learning*.
- Lista de [grupos de discussão](#) de Python, por estado.
- [GitHub](#) é um fórum, uma comunidade de programadores, onde os participantes inserem programas. Lá se encontra na íntegra o livro *Python Data Science Handbook* de Jake VanderPlas. {LF}
- [Eventos sobre Python](#)

20. AGRADECIMENTOS

- Colaborações (fazer uma busca na página com as abreviaturas): {AL} Alexandre Leite; {LF} Luís Felipe Carvalho; {MDG} Marco Dimas Gubitoso; {PC} Paulo César Zandoná Vieira achou 2 errinhos. Eduardo Furlan (EF) fez inúmeras adições de descrições de funções, adições no texto e correções ortográfica; foram anotadas as funções contribuídas por ele (com minha revisão).